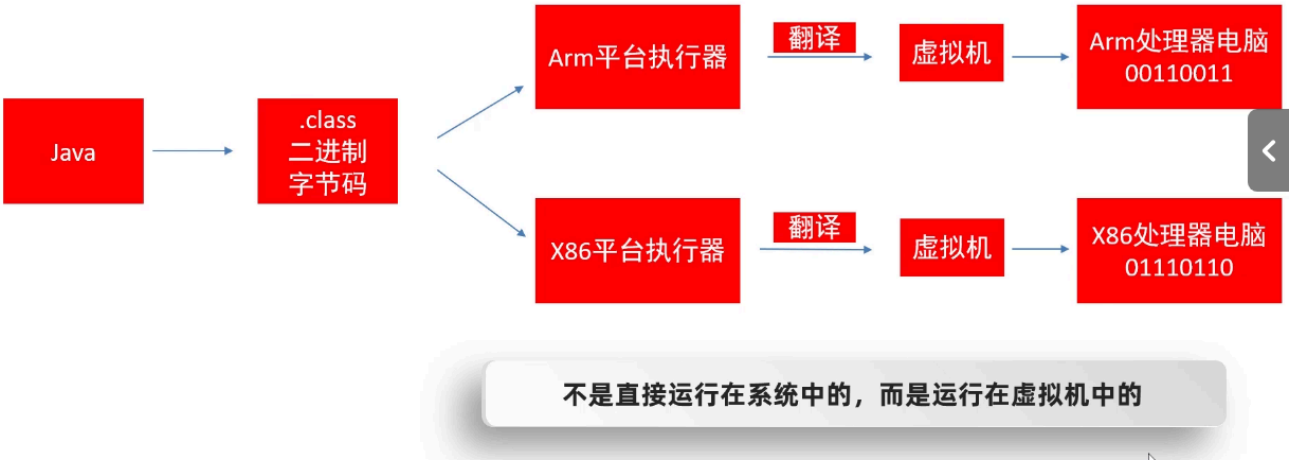
Java学习

Java是一门混合型语言，编译器将java文件编译为.class二进制字节码。



JDK：java开发工具包

JVM虚拟机：Java程序运行的地方

核心类库：Java已经写好的东西，直接使用

开发工具：javac,java,jdb,jhat.....

JRE:java的运行环境

JVM 核心类库 运行工具

IDEA项目结构

project（项目）

module（模块）

package(包)

class（类）

进制知识

二进制：由0和1组成，代码中以0b开头

十进制：由0~9组成，前面不加任何前缀

八进制：由0~7组成，代码中以0开头

十六进制：由0~9还有a~f组成，代码中以0x开头

任意进制转10进制

系数*基数的权次幂 相加

10进制转其他进制

除基取余法

不断除以基数（几进制基数就是几），得到余数，直到商为0，再将余数倒着拼起来即可。

基本数据类型

整数　byte　-128~127　short　-32768~+32767　int 10位数 long 19位数

浮点数　float　double

字符　char　0~65535

布尔　boolean true/false

标识符

给类，方法，变量等起的名字

命名规则：由数字，字母，下划线和$组成

不能以数字开头

不能是关键字

区分大小写

小驼峰命名：方法，变量

标识符是一个单词的时候，全部小写

name

标识符由多个单词组成的时候，第一个单词首字母小写，其他单词首字母大写

firstName

大驼峰命名法：类名

一个单词首字母大写　Student

多个单词组成：每个单词的首字母大写

GoodStudent

键盘录入

```
import java.util.Scanner;//导包，看Scannner这个类在哪里
Scannner sc=new Scanner(System.in);//创建对象，sc是变量
int i=sc.nextInt();//i是变量
```

"+"的用法

隐式转换：将一个取值范围小的数值，转换成取值范围大的数据

byte<short<int<long<float<double

大转小：强制类型转换

'+'出现在字符串中会将字符串进行拼接

一.数组

数组是一种容器，可以用来存储同种数据类型的多个值

静态初始化：手动指定数组元素，系统会根据元素个数，计算出数组的长度

double[] array2=new double[]{11.1,22.2,33.3};

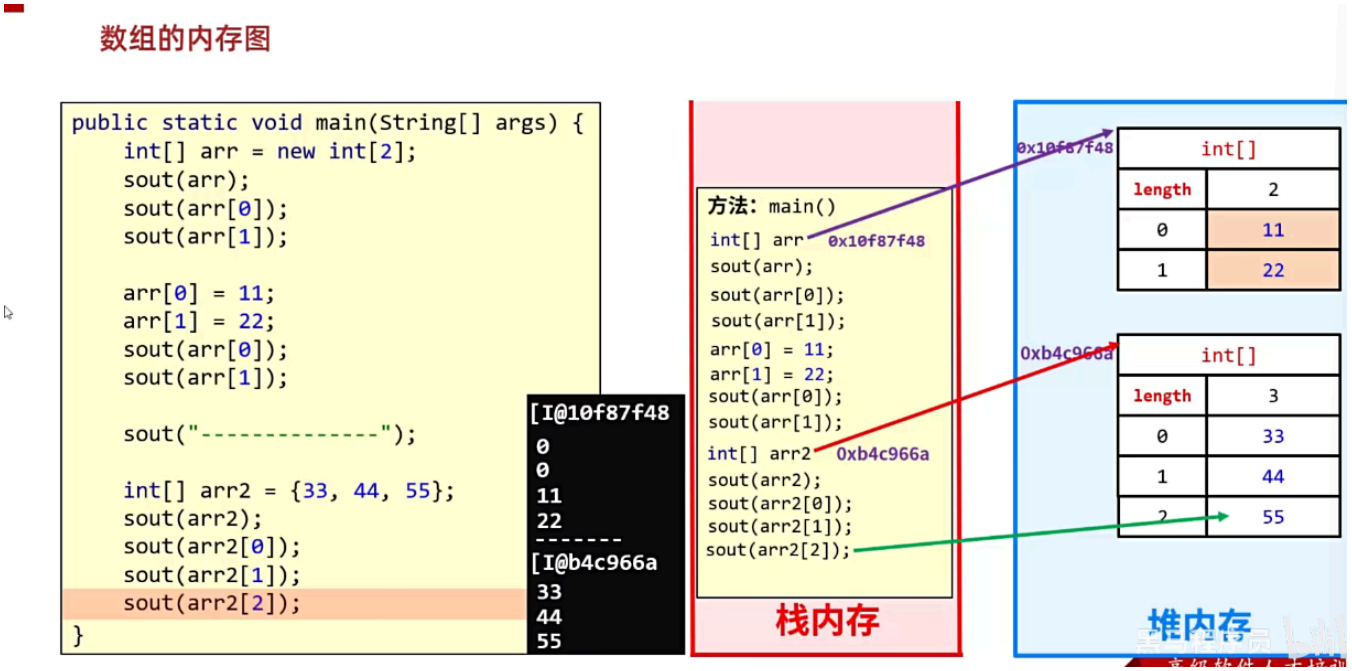
动态初始化：手动指定数组长度，由系统给出默认初始化值

int[] array =new int [3];

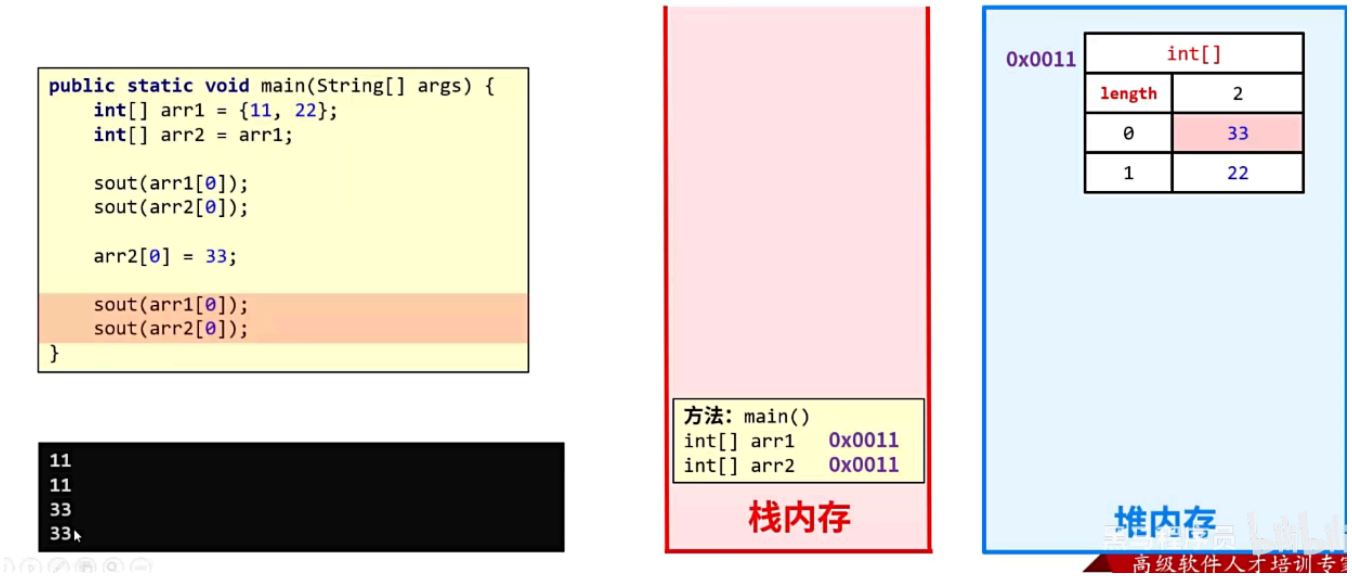
数组的内存图

数组变量是引用数据类型，在栈内存中，记录堆内存中数组对象的地址值。通过索引去找对应的值

数组对象被创建在堆内存中



两个数组指向同一个空间的内存图



二.方法

方法是程序中最小执行单元

可以将重复具有独立功能的代码封装，类似函数

2.1 方法的定义

定义格式

```
public static void 方法名(){
    方法体（打包起来的代码）;
}
```

方法调用

```
方法名();
```

带参数的方法定义和格式

类似函数

带返回值的方法定义和调用

```
public static 返回值类型 方法名 (参数){
    方法体;
    return 返回值;
}
```

2.2 方法的重载

在同一个类中，方法名相同，参数不同的方法。与返回值无关。

2.3 方法的内存

Java内存分配

栈　方法运行时使用的内存，方法进栈运行，运行完毕出栈

堆　new出来，在堆内存中开辟了一个小空间

方法区　存储可以运行的class文件

本地方法栈　JVM使用操作系统功能的时候使用

寄存器　给CPU使用

基本数据类型

整数 浮点数 布尔 字符

数据存储在自己的空间中

引用数据类型

变量中存储的不是真实值，而是地址值，相当于一个索引

数据值存储在其他空间中，自己空间存储地址值

2.4 方法的值传递

传递引用数据类型时传递的是地址值，形参的改变影响实际参数的值

基本数据类型传递的时候传递的是值，在方法中重新开辟一个空间，将值copy到这个新空间中

三.面向对象

3.1 类和对象

类：是对象共同特征的描述

对象:真实存在的具体实例，可以使用对象来完成需要的一些功能

类的定义

```
Public class 类名{
    1.成员变量（代表属性，一般是名词）
    2.成员方法（代表行为，一般是动词）
    3.构造器
    4.代码块
    5.内部类
}

public class phone{
    //属性
    String brand;
    double price;

    //行为
    public void call(){
```

```
    }
    public void playGame(){

    }
}
```

得到类的对象

类名+对象名=new+类名();

Phone P=new Phone();

如何得到对象?

先创建类，然后利用类创建对象

```
类名 对象名=new 类名();
```

对象的作用

对象.成员变量 可以用于访问对应的成员变量和进行修改

对象.成员方法 可以调用对象中的方法

定义类的补充说明

- Javabean类：用来描述一类事物的类

  在该类中不写main方法
- 测试类：编写main方法的类

可以在测试类中创建javabean类的对象并进行赋值调用

- 一个Java文件中可以定义多个class类，且只能一个类是public修饰，并且public修饰的类名必须成为代码文件名
- 在类中定义成员变量时一般无需初始化，存在默认值

3.2 封装

对象代表什么，就得封装对应的数据，并提供数据对应的行为

关键字 `private`:权限修饰符　　可以修饰成员（成员变量和成员方法）　　被private修饰的成员只能在本类中才能访问

```
public class Student{
    private String name;
```

```
    private int age;
    private String gender;

    //构造一个方法保证数据输入的正确性
    public void SetAge(int a){
        if(a>=18&&a<=50){
            age=a;
        }
        else{
            system.out.println("非法数据");
        }
    }

    public int getAge(){
        return age;
    }
}
```

特殊类

Integer:int的包装类 用对象的形式表示整数 初值为Null 避免直接使用int导致被赋予初值0

3.3 就近原则和this关键字

成员变量：定义在class里面，方法的外面

局部变量：定义在方法里面

就近原则：程序语句离哪个近就使用哪个变量

`this`关键字可以规定这个位置的变量使用成员变量

```
public class Student{
    private int age;
    public void method(){
        int age=10;
        system.out.println(this.age);//age使用成员变量age
    }
}
```

3.4 构造方法

作用：在创建对象的时候给成员变量进行赋值

构造方法的格式

```
public class student{
    修饰符 类名(参数){
        方法体;
    }
}
```

方法名与类名相同，大小写也要一致

没有返回值类型，不写void

没有具体的返回值

执行时机

- 创建对象的时候由虚拟机调用，不能手动调用构造方法
- 每创建一次对象，就会调用一次构造方法

空参构造只有虚拟机赋的初值

含参构造可以在初始化的时候直接赋值成员变量

```
public class Student{
    private String name;
    private int age;

    public Student(){
        System.out.println("xxx");
    }
    public Student(String name,int age){
        this.name=name;
        this.age=age;
    }
}
```

3.5 标准JavaBean的构造

要求

- 类名需要见名知意
- 成员变量使用private修饰
- 提供至少两个构造方法
    - 无参构造
    - 带部参数构造
- 成员方法
    - 提供每一个成员变量对应的Set()和get()方法
    - 其他的行为

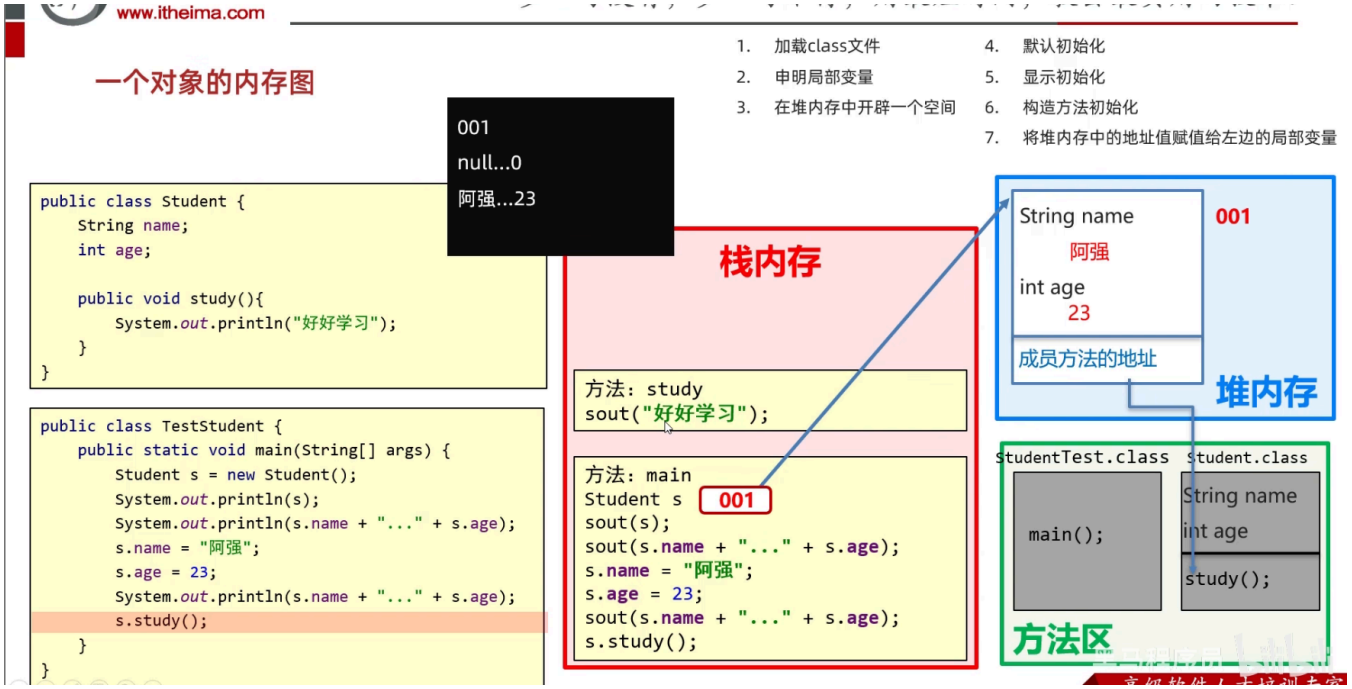
3.6 Java内存分配

字节码文件加载时进入方法区

方法运行时所进入的内存+变量所在的内存->栈内存

new出来的东西在这块内存中开辟空间并产生地址->堆内存

一个对象的内存图



3.6 抽象类

定义格式

```
//抽象方法
    public abstract 返回值类型 方法名（参数列表）;
//抽象类
    public abstract class 类名{}
```

- 抽象类不能实例化
- 抽象类中不一定有抽象方法，有抽象方法的类一定是抽象类
- 可以有构造方法
- 抽象类的子类
    - 重写抽象类中的所有抽象方法
    - 抽象类

3.7 接口

接口就是一种规则，是对行为的抽象

接口的定义

```
public interface 接口名{

}
```

- 接口不能实例化，不能创建接口的对象

- 接口和类之间是实现关系，通过implements关键字表示

  public class+类名+implements 接口名{}

- 接口的子类（实现类）

  要么重写接口中的所有抽象方法

  要么是抽象类

- 接口和类是实现关系，类可以单实现接口，也可以多实现接口
  - public class 类名+implements+接口名1，接口名2{}
- 实现类还可以在继承一个类的同时实现多个接口
  - public class 类名+extends+父类+implements+接口名1，接口名2{}

3.7.2 接口中成员的特点

- 成员变量：只能是常量

  默认修饰符 public static final

- 构造方法：无
- 成员方法：抽象方法 默认修饰符：public abstract

接口和类之间的关系

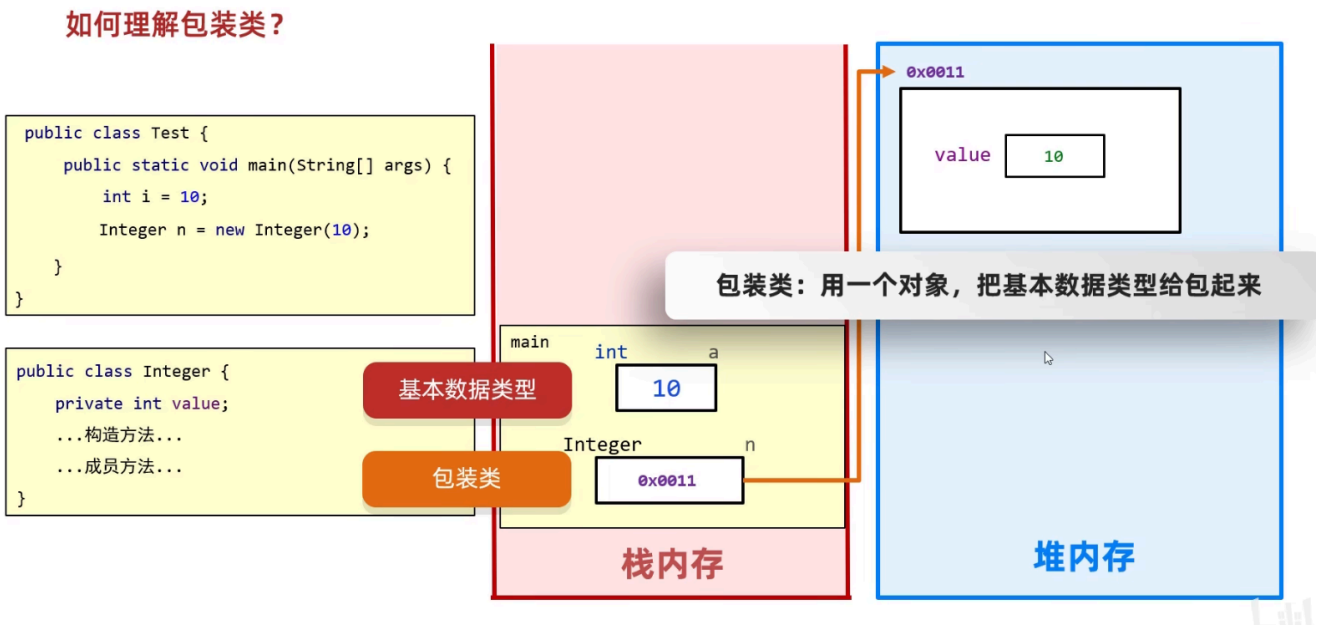
- 类和类之间的关系：继承关系，只能单继承，不能多继承，但是可以多层继承
- 类和接口的关系：实现关系，可以单实现，也可以多实现，还可以在继承一个类的同时实现多个接口
- 接口和接口的关系：继承关系，可以单继承，也可以多继承

3.7.3 接口的应用

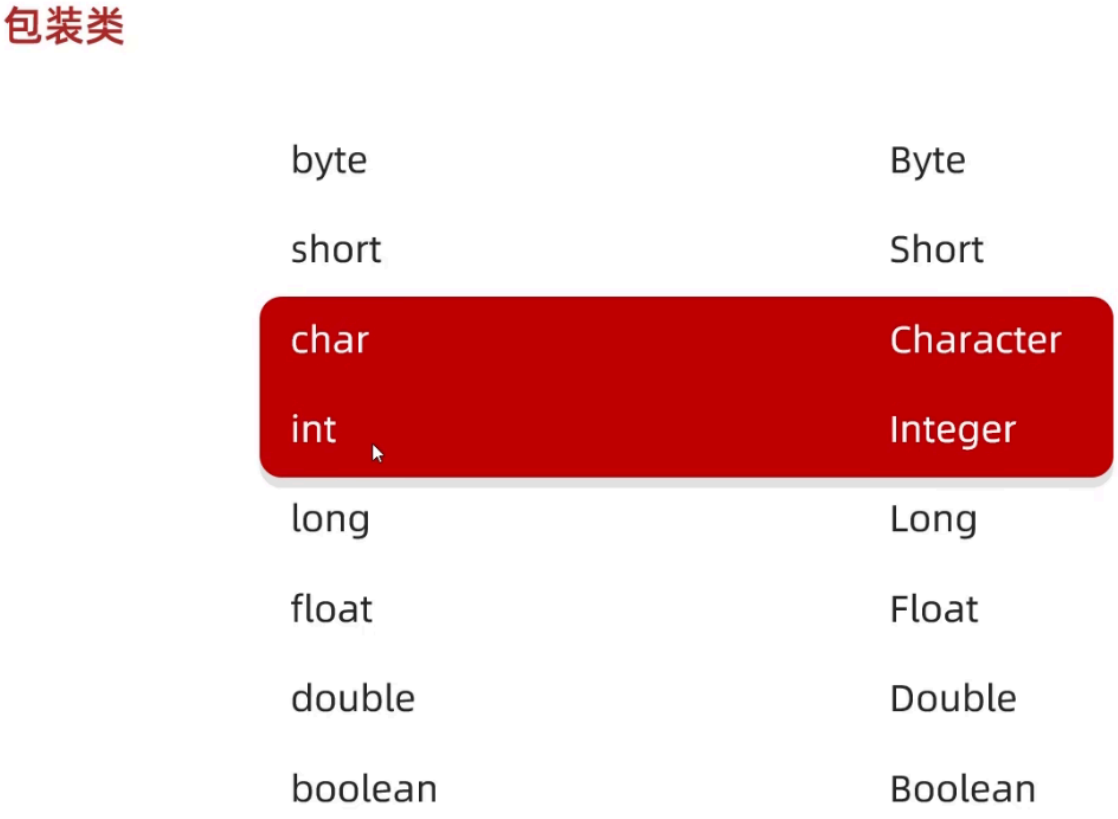
- 接口代表规则，是行为的抽象。想要让哪个类拥有一个行为，就让这个类实现对应的接口就可以了
- 当一个方法的参数是接口时，可以传递接口所有实现类的对象，这种方式称之为接口多态。

3.8 包装类

基本数据类型对应的引用类型，本质上是用一个对象，把基本数据类型给包起来



八种基本数据类型对应的包装类



JDK5以后实现了自动装箱（自动将基本数据类型变成对象）和自动拆箱（自动将对象变成基本数据类型）

`Integer i=10;`直接赋值运算即可

3.9 内部类

- 一个类里面再定义一个类

```
public class Outer（外部类）{
    public class Inner（内部类）{

    }
}
```

- 内部类表示的事物是外部类的一部分
- 内部类单独出现没有任何意义
    - 内部类的访问特点
        - 内部类可以直接访问外部类的成员，包括private
        - 外部类要访问内部类的成员，必须创建对象
- 使用的条件：B类表示的事物是A类的一部分，且B类单独存在没有意义

3.9.1 成员内部类

3.9.2 静态内部类和局部内部类

3.9.3 匿名内部类

- 隐藏了名字的内部类（java自己默认取名）

```
new 类名或者接口名(){
    重写方法;
};
```

包含继承/实现 方法重写 创建对象

如果是new+接口名 相当于是new了一个没有名字的类去实现这个接口

new+类名（A) +{} {}中没有名字的类是A类的子类

- 可以写在成员位置，也可以写在局部位置

四.字符串

4.1 String概述

注意点

字符串的内容不会改变，它的对象在创建后不能被更改

创建String对象的两种方式

- 直接赋值
- new

4.2 JAVA的常见方法

- 比较

  boolean equals(要比较的字符串)

boolean equalsIgnoreCase(要比较的字符串) 忽略大小写的比较

4.3 Static关键字

static表示静态，是java中的一个修饰符，可以修饰成员方法，成员变量

静态变量

被static修饰的成员变量是静态变量 放在堆内存中的静态存储位置

静态变量随着类的加载而加载，优先于对象出现

在堆内存中是共享的，多个对象可以使用同一个静态变量，谁需要用谁去拿

静态方法

- 多用在测试类(用来检查其他类是否书写正确，含main方法）和工具类（帮我们做一些事情但不描述任何事物的类)中
- 类名见名知意
- 私有化构造方法
- 方法定义为静态

```
public class ArrUtil {

    private ArrUtil() {}//私有化构造方法，防止其它类中创建对象，创建工具类的对象没有意义

    public static int getMax(...) {...}

    public static int getMin(...) {...}

    public static int getSum(...) {...}

    public static int getAvg(...) {...}
}
```

static的注意事项

- 静态方法中只能访问静态
- 非静态方法可以访问所有
- 静态方法中没有this关键字

4.4 final关键字

final修饰方法，类，变量

- 修饰方法：方法是最终方法，不能被重写
    - 一般当前方法是一种规则，不能被改变时可以使用final
- 修饰类：该类是最终类，不能被继承
- 修饰变量：叫做常量，只能被赋值一次

细节

final修饰的变量是基本类型：变量存储的数据值不能发生改变

final修饰的变量是引用类型：变量存储的地址值不能发生改变，对象内部可以改变

4.5 权限修饰符

缺省/默认

有四种作用范围由小到大（ **private** < **空着不写** < **protected** < **public** ）

修饰符	同一 个类中	同一个包中 其他类	不同包下的 子类	不同包下的 无关类
private	√			
空着不写	√	√		
protected	√	√	√	
public	√	√	√	√

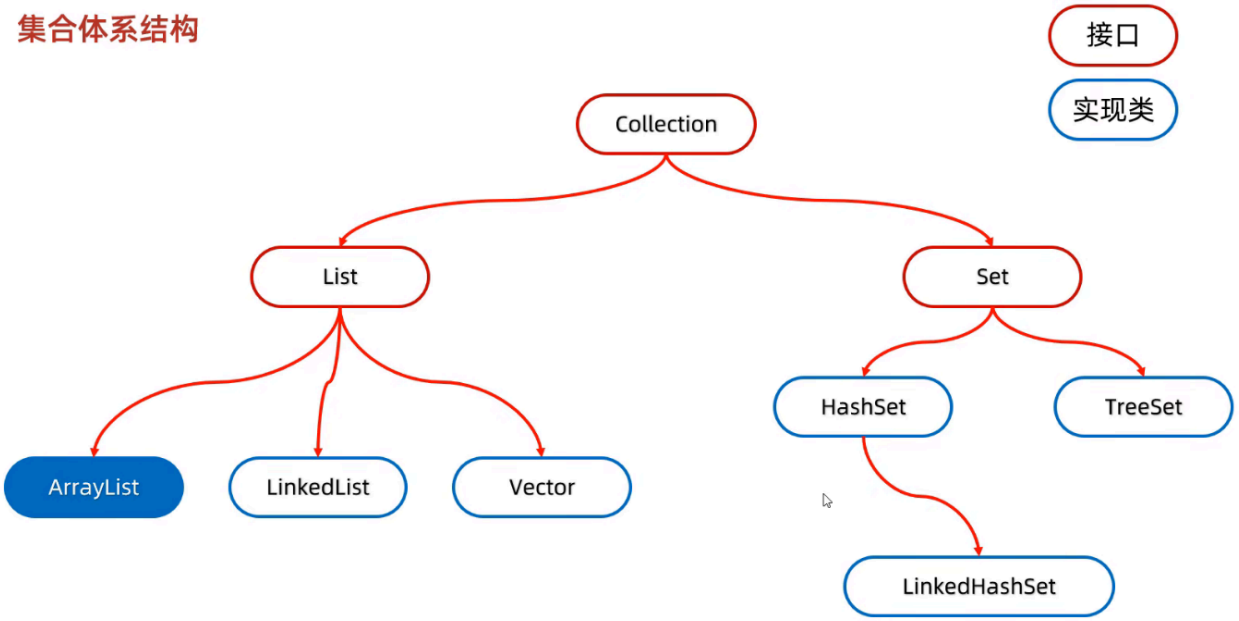
- 成员变量一般私有
- 方法一般公开
    - 如果方法中的代码是抽取其他方法中的共性代码，这个方法一般也私有

五.集合

集合长度可变，可以存引用数据类型和基本数据类型(必须包装类)

集合体系结构

集合体系结构



List系列集合：添加的元素是有序，可重复，有索引的

有序：存和取的顺序是一样的

可重复：集合中存取的元素是可以重复的

有索引：可以通过索引访问

Set系列集合

添加的元素是无序，不重复，无索引的

5.1 Arraylist

```
ArrayList<String> list=new ArrayList<>();
```

增删改查

方法名	说明
boolean add(E e)	添加元素，返回值表示是否添加成功
boolean remove(E e)	删除指定元素,返回值表示是否删除成功
E remove(int index)	删除指定索引的元素,返回被删除元素
E set(int index,E e)	修改指定索引下的元素,返回原来的元素
E get(int index)	获取指定索引的元素
int size()	集合的长度，也就是集合中元素的个数

5.2 Collection

- 单列集合的祖宗接口，它的功能是全部单列集合都可以继承使用的

方法名称	说明
public boolean add(E e)	把给定的对象添加到当前集合中
public void clear()	清空集合中所有的元素
public boolean remove(E e)	把给定的对象在当前集合中删除
public boolean contains(Object obj)	判断当前集合中是否包含给定的对象
public boolean isEmpty()	判断当前集合是否为空
public int size()	返回集合中元素的个数/集合的长度

5.3 Collection 遍历

5.3.1 迭代器遍历

- 迭代器不依赖索引
- Iterator 对象

六.继承

关键字 `extends` 可以让一个类和另一个类建立起继承关系

```
public class Student extends Person{}
```

Student 称为子类 (派生类)　　Person称为父类（基类/超类）

6.1 继承基础

使用继承的时间

当类与类之间存在相同的内容，并满足子类是父类中的一种，可以考虑使用继承来优化代码

继承的特点

Java支持单继承，不支持多继承（一个子类对应一个父类），但支持多层继承

比如子类A继承父类B，父类B继承父类C　每一个类都直接或间接的继承于object

继承的设计

画图法

将几个不同的类画出来，提取其中相同的特点向上总结画图，然后再次提取新的共同特点，向上总结抽取。

写代码从上往下，最上面为父类，越往下是子类。

6.2 继承的内存

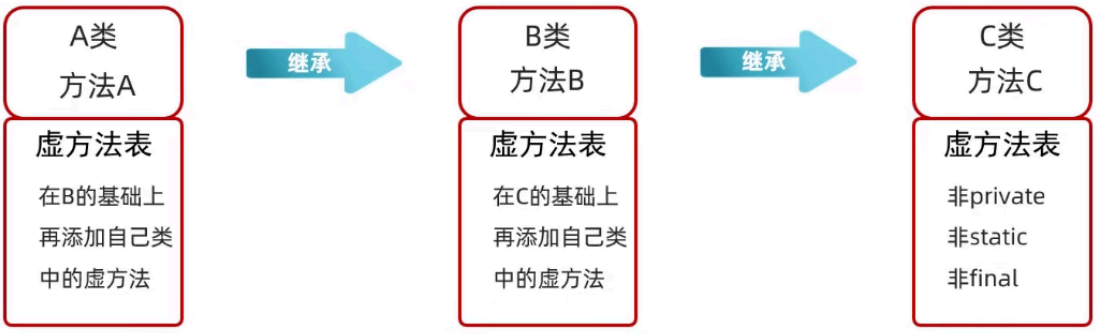
子类继承父类中的哪些内容

构造方法 不管什么修饰符都不能继承

成员变量·　　非私有 可以继承　private可以继承

成员方法 非私有 可以继承　private不能继承

- 成员变量：private虽然可以继承但是不能直接使用
- 成员方法：



只有父类中的虚方法（非private 非static 非final)才能被子类继承

6.3 继承中的特点

6.3.1 成员变量和方法的访问特点

super关键字：使用父类中的成员变量或者成员方法

6.3.2 方法的重写

当父类的方法不能满足子类现在的需求时，需要进行方法重写

当子类中出现了和父类一样的方法声明，称该子类这个方法是重写的方法

@Override

- 该注解放在重写后的方法上，校验子类重写时语法是否正确
- 加上注解后如果有红色波浪线，表示语法错误

本质上是虚方法表中对方法重写后覆盖了原有虚方法表中的方法

注意事项

1. 重写方法的名称、形参列表必须与父类中的一致。
2. 子类重写父类方法时，访问权限子类必须大于等于父类 （暂时了解： 空着不写 < protected < public）
3. 子类重写父类方法时，返回值类型子类必须小于等于父类
4. **建议：重写的方法尽量和父类保持一致。**
5. 私有方法不能被重写。
6. 子类不能重写父类的静态方法，如果重写会报错的。

6.3.3 构造方法的访问特点

- 父类中的构造方法不会被子类继承
- 子类中所有构造方法默认先访问父类中的无参构造，再执行自己
- 子类初始化之前，一定要调用父类构造方法先完成父类数据空间的初始化
- 子类构造方法第一行默认是：super()，不写也存在，且必须在第一行
- 如果想要调用父类的有参构造，必须手动写super进行调用

```
public student(String name,int age){
    super(name,age);//调用父类中的含餐构造
}
```

6.3.4 this,super使用总结

this:一个变量，表示当前方法调用者的地址值

super:代表父类存储空间

关键字	访问成员变量	访问成员方法	访问构造方法
this	this.成员变量 访问本类成员变量	this.成员方法(...) 访问本类成员方法	this(...) 访问本类构方法
super	super.成员变量 访问父类成员变量	super.成员方法(...) 访问父类成员方法	super(...) 访问父类构造方法

七.多态

- 什么是多态：同类型的对象，表现出的不同形态
- 多态的表现形式:

```
父类类型 对象名称=子类对象;
```

- 多态的条件:

    - 有继承实现关系
    - 有父类引用指向子类对象
    - 用方法重写
- 多态的好处：

  使用父类型作为参数，可以接收所有子类对象，体现多态的扩展性与便利性
- 多态调用成员的特点：
    - 调用成员变量时，编译看左边，运行也看左边
        - 编译时在父类里面找变量，运行时也使用父类里面的变量
    - 调用成员方法时，编译看左边，运行看右边
        - 看左边指的是在父类中去找方法，实际运行使用子类中的方法
- 多态的优势
    - 多态形式下，赋值时右边对象可以变更，使得业务逻辑变化时后续的代码无需修改
    - 定义方法时，使用父类型作为参数，可以接收所有的子类对象
- 多态的弊端：调用成员方法时编译看左边，因此不可以使用子类的特有功能，会报错
- 强制类型转换解决的问题：
    - 可以转换成真正的子类类型，从而调用子类独有功能

七.包

包是文件夹，用来管理各种不同功能的java类，方便后期代码维护

- 包名的规则：公司域名反写+包的作用，需要全部英文小写，见名知意

  com.example.[作用]
- 类的全类名：包名+类名

使用其他类的原则

- 使用同一个包中的类时，不需要导包（默认在本包中寻找）
- 使用java.lang包中的类不需要导包
- 其他情况均需导包
- 如果两个包中的类同名，需要使用全类名

八.Lambda表达式

```
()->{


}
```

( )对应方法的形参

-> 固定格式

{}对应着方法的方法体

Lambd a表达式可以用来简化函数式接口的匿名内部类的写法

函数式接口：有且仅有一个抽象方法的接口叫做函数式接口，接口上方可以加@FunctionalInterface注解进行是否满足要求的验证

使用案例

- 利用匿名内部类的形式调用方法：调用方法的时候，如果方法的形参是一个接口，那么形参需要传递这个接口的实现类对象。如果实现类对象只要用到一次，可以使用匿名内部类的形式进行书写

```
method(new swim(){
    @Override
    //重写方法
    public void swimming(){
        System.out.println("xxxx");
    }
});

public static void method(Swim s){
    s.swimming();
}
interface Swim{
    public abstract void swimming();
}
```

```
//利用Lambda表达式进行改写
method(
    ()->{
          System.out.println("xxxx");
        }
);

public static void method(Swim s){
    s.swimming();
}

interface Swim{
    public abstract void swimming();
}
```

- lambda的省略规则：
  - 参数类型可以省略不写
  - 如果只有一个参数，参数类型可以省略，同时()也可以省略
  - 如果lambda表达式的方法体只有一行，大括号，分号，return可以省略不写，但是需要同时省略

九.异常

- 异常：代表程序出现的问题